CSA4704-DEEP LEARNING

NAME:HARITHI S **REG NO:192421151**

Experiment 15: BPTT Flow and Gradient Analysis

Problem Statement: A stock-analytics firm finds its RNN ignores price movements from more than a few days ago. The team suspects the gradients are vanishing. Daily closing prices of a stock index over 3 years, with sequence lengths of 10, 50 and 100 days. The objective is to trace back propagation through time and measure gradient magnitude at each time step. Use TensorFlow, Keras, NumPy, Matplotlib.

Tasks:

1. Build a SimpleRNN and use tf.GradientTape to compute gradients of the loss.
2. Record the gradient norm with respect to the hidden state at each time step.
3. Plot gradient norm against time step for each sequence length.
4. Compare tanh and ReLU activations.
5. Explain vanishing and exploding gradients from the plots.

FORMULA:

1. Recurrent Neural Network Hidden State

$h_t = f(W_x x_t + W_h h_{t-1} + b_t)$

2. Tanh Activation Function

$\tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$

3. ReLU Activation Function

$\text{ReLU}(x) = \max(0, x)$

4. Loss Function (Mean Squared Error)

$L = (1/N) \Sigma (y_i - \hat{y}_i)^2$

5. Backpropagation Through Time (BPTT)

$\partial L/\partial h\square = (\partial L/\partial h\square_{+1})(\partial h\square_{+1}/\partial h\square) + \partial L\square/\partial h\square$

6. Gradient Norm

$\|\nabla h\square\ L\|_2 = \sqrt{\Sigma_i}\ (\partial L/\partial h\square_{,i})^2$

7. Gradient Propagation Across Time Steps

$\partial h\square_{+}\square/\partial h\square = \prod\square_{=}\square_{+1}{}^{t+k}\ (\partial h\square/\partial h\square_{-1})$

8. Vanishing Gradient

$\|\nabla h\square\ L\| \rightarrow 0$

9. Exploding Gradient

$\|\nabla h\square\ L\| \rightarrow$ very large values

10. Root Mean Squared Error (RMSE)

$RMSE = \sqrt{[(1/N)\ \Sigma\ (y_i - \hat{y}_i)^2]}$

CODE:

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

np.random.seed(42)
tf.random.set_seed(42)

# Generate 3 years of stock price data
prices = 100 * np.exp(np.cumsum(np.random.normal(0, 0.01, 1095)))
prices = ((prices - prices.mean()) / prices.std()).astype(np.float32)

sequence_lengths = [10, 50, 100]
activations = ["tanh", "relu"]
results = {}

# Prepare sequences and train RNN
for activation in activations:
    for length in sequence_lengths:
```

```
X = np.array([prices[i:i+length]
              for i in range(len(prices)-length)])
y = prices[length:]

X = X[..., np.newaxis]
split = int(0.8 * len(X))
X_train, y_train = X[:split], y[:split]
X_test, y_test = X[split:], y[split:]

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(length, 1)),
    tf.keras.layers.SimpleRNN(
        16, activation=activation, return_sequences=True
    ),
    tf.keras.layers.Dense(1)
])

model.compile(optimizer="adam", loss="mse")
model.fit(X_train, y_train, epochs=3,
          batch_size=32, verbose=0)

# Extract trained weights
rnn = model.layers[0]
Wxh, Whh, bh = rnn.get_weights()

# Calculate gradients using BPTT
sample = tf.constant(X_test[:1])
states = []

with tf.GradientTape() as tape:
    h = tf.zeros((1, 16))

    for t in range(length):
        h = sample[:, t, :] @ Wxh + h @ Whh + bh
        h = tf.math.tanh(h) if activation == "tanh" else tf.nn.relu(h)
        states.append(h)

    prediction = model.layers[-1](states[-1])
    loss = tf.reduce_mean((prediction - y_test[:1]) ** 2)
```

```
        gradients = tape.gradient(loss, states)
        norms = [float(tf.norm(g).numpy()) for g in gradients]
        results[(activation, length)] = norms

# Plot gradient norms
fig, axes = plt.subplots(2, 3, figsize=(14, 7))

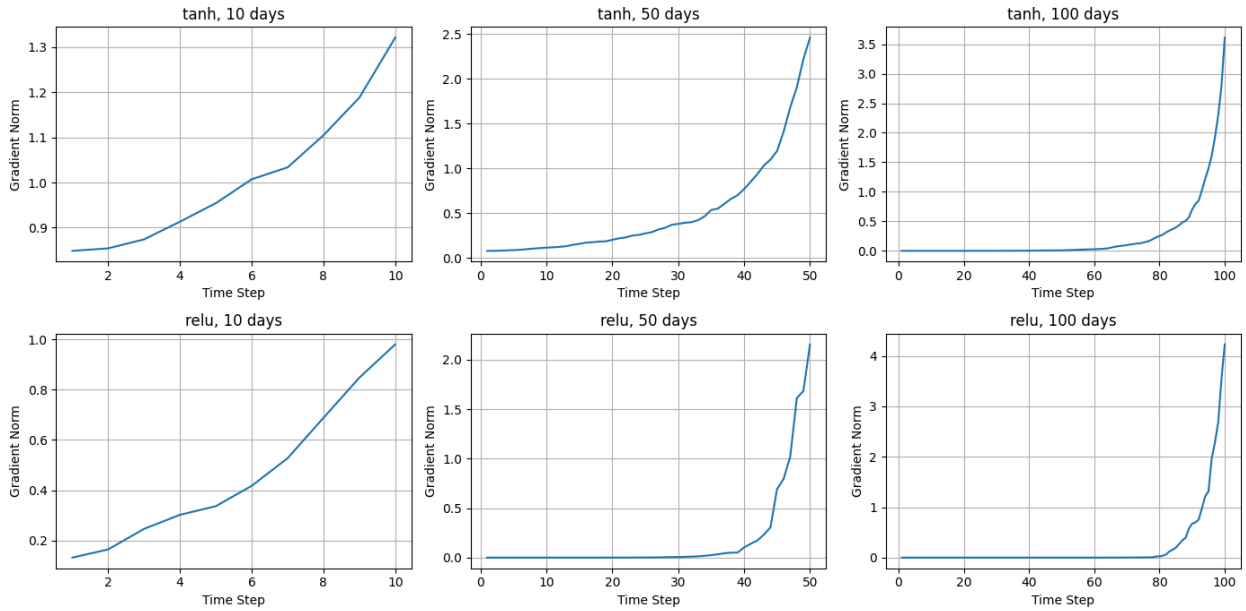
for i, activation in enumerate(activations):
    for j, length in enumerate(sequence_lengths):
        axes[i, j].plot(
            range(1, length + 1), results[(activation, length)]
        )
        axes[i, j].set_title(f"{activation}, {length} days")
        axes[i, j].set_xlabel("Time Step")
        axes[i, j].set_ylabel("Gradient Norm")
        axes[i, j].grid(True)

plt.tight_layout()
plt.show()

# Display results
for key, values in results.items():
    print(key, "Mean gradient norm:", np.mean(values))

print("Experiment completed successfully.")
```

OUTPUT:



('tanh', 10) Mean gradient norm: 1.0099932551383972
('tanh', 50) Mean gradient norm: 0.5153497651219368
('tanh', 100) Mean gradient norm: 0.24450562586309388
('relu', 10) Mean gradient norm: 0.4639996141195297
('relu', 50) Mean gradient norm: 0.18394443429594504
('relu', 100) Mean gradient norm: 0.22609213226578742
Experiment completed successfully.